

Parallel Algorithms and Multithreading	
Hands-on Activity 13.1	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: Nov 4, 2025
Section: CPE21S4	Date Submitted: Nov 4, 2025
Name(s): Punay, Heidee S.	Instructor: Engr. Jimlord Quejado
A. Output(s) and Observation(s)	
Simple One-Threaded Example	
Screenshot	main.cpp <pre>1 #include <iostream> 2 #include <thread> 3 4 5- void print(int n, const std::string &str) { 6 std::cout << "Printing integer: " << n << std::endl; 7 std::cout << "Printing string: " << str << std::endl; 8 } 9 10- int main() { 11 std::thread t1(print, 10, "T.I.P."); 12 t1.join(); 13 return 0; 14 }</pre> Output Printing integer: 10 Printing string: T.I.P. === Code Execution Successful ===
Analysis	In this part, the function called print that shows a number and a word. Then a thread that is named t1 to run the print function. The thread runs a task while the main program waits. t1.join() tells the program to wait for the thread to finish before proceeding to the next.

Screenshot

main.cpp

```
1 #include <iostream>
2 #include <pthread.h>
3 #include <cstring>
4
5 char messages[4][20] = {
6     "T.I.P.",
7     "Competent",
8     "Computer",
9     "Engineers"
10 };
11
12 void* print(void* arg) {
13     int index = *(int*)arg;
14     std::cout << index << " : " << messages[index] << std::endl;
15     return nullptr;
16 }
17
18 int main() {
19     pthread_t threads[4];
20     int indices[4] = {0, 1, 2, 3};
21
22     for (int i = 0; i < 4; i++) {
23         pthread_create(&threads[i], nullptr, print, &indices[i]);
24     }
25
26     for (int i = 0; i < 4; i++) {
27         pthread_join(threads[i], nullptr);
28     }
29
30     return 0;
31 }
```

Output:

Output

```
0 : T.I.P.
2 : Computer
1 : Competent
3 : Engineers
```

=== Code Execution Successful ===

Analysis

In this part, there are several threads are created. Each thread printed different words like the "T.I.P" or "Computer". Stored them in a list to start working. The loops are for waiting for each one to finish using join() . This program shows that it can run many tasks at the same time.

B. Answers to Supplementary Activity

Part A:

1. Write a definition of multithreading and its advantages/ disadvantages.

Multithreading is a technique in which a program is divided into smaller units of execution, called threads. Each thread runs independently but shares resources such as memory, allowing tasks to run simultaneously.

Advantages:

- Improved performance

- Responsiveness
- Better resource utilization
- Simplified Modeling
- parallelism

Disadvantages:

- Complexity
- Synchronization overhead
- Context Switching
- Hard to predict behavior
- Limited by hardware

2. Rationalize the use of multithreading by providing at least 3 use cases.

In a web server, multithreading can be used to handle multiple requests from different clients concurrently. Each client request can be processed in its own thread, improving the server's ability to handle simultaneous connections.

3. differentiate between parallelism and concurrency

Parallelism executes multiple tasks at the same time on multiple processing units. In parallelism, tasks are divided into smaller sub-tasks that are processed seemingly simultaneously or in parallel. Concurrency deals with multiple tasks at once. Tasks make progress by sharing time on the same processing resource.

Part B:

main.cpp



```
2  #include <thread>
3  using namespace std;
4
5  int globalVar = 0;
6
7  int main() {
8      cout << "Before join t1" << endl;
9
10     thread t1([] {
11         globalVar += 5;
12         cout << "Global variable after t1: " << globalVar << endl;
13     });
14     t1.join();
15     cout << "After join t1" << endl;
16
17     thread t2([] {
18         globalVar += 10;
19         cout << "Global variable after t2: " << globalVar << endl;
20     });
21     t2.join();
22     cout << "After join t2" << endl;
23
24     thread t3([] {
25         globalVar += 15;
26         cout << "Global variable after t3: " << globalVar << endl;
27     });
28     t3.join();
29     cout << "After join t3" << endl;
30
31     return 0;
32 }
```

Output:

Output

```
Before join t1
Global variable after t1: 5
After join t1
Global variable after t2: 15
After join t2
Global variable after t3: 30
After join t3

=== Code Execution Successful ===
```

C. Conclusion & Lessons Learned

To conclude, I learned that multithreading means that it runs many tasks at once inside just one program. It's useful, like in games, might use one thread for graphics and another for sound. I also learned the difference between concurrency and parallelism. It's like having one coffee machine used by two people versus two machines used by two people. In the supplementary activity, in the global variable example, each thread is created directly inside the main() function using a lambda expression, which is a short block of code that performs a task without needing a separate function. These threads each add a different value—5, 10, and 15—to a shared variable called globalVar. After starting each thread, the program uses join() to make sure that the thread finishes before moving on to the next one.

D. Assessment Rubric

E. External References

GeeksforGeeks. (2025e, October 3). Multithreading in C++. GeeksforGeeks.

<https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/>

Best online IT training & certification platform - Multisoft. (n.d.). Multisoft Virtual Academy.

<https://www.multisoftvirtualacademy.com/blog/common-advantages-and-disadvantages-of-multithreading-in-java>

GeeksforGeeks. (2025d, August 7). Difference between Concurrency and Parallelism. GeeksforGeeks.

<https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/>